

广州软件学院
GUANGZHOU UNIVERSITY OF SOFTWARE

实验报告封面

课程名称： Python Web 企业级项目开发 **课程代码：** SK3017

任课老师： 陈广豪 **实验指导老师:** 陈广豪

实验报告名称： 课程设计

学生姓名： 李宇恒

学号： 2520148114

教学班：

递交日期： 2026.10.27

签收人： 陈广豪

我申明，本报告内的实验已按要求完成，报告完全是由我个人完成，并没有抄袭行为。我已经保留了这份实验报告的副本。

申明人(签名):李宇恒

实验报告评语与评分：

评阅老师签名：

一、课程名称：Python Web 企业级项目开发

二、实验日期：2026 年 10 月 27 日

三、实验目的：

深本课程设计旨在通过**企业级项目实战**，全面检验与巩固学生在《Python Web 企业级项目开发》课程中所学的理论知识与实践技能。学生需在**智能租房项目**与**博客系统项目**中任选其一，独立或分组完成从需求分析、系统设计、编码实现到测试部署的全过程。通过本设计，学生应能：

1. **综合运用 Flask 框架核心技术**，包括路由、模板、表单、数据库 ORM、蓝图模块化等；

2. **掌握前后端不分离架构下的开发流程**，实现数据动态渲染与用户交互；

3. **熟练使用 MySQL 进行数据建模与管理**，并掌握基本的数据查询与事务处理；

4. **初步接触数据可视化技术**，能使用 ECharts 实现图表展示；

5. **培养项目文档撰写、代码调试与系统测试能力**，提升工程化思维与独立开发能力。

实验用的仪器和材料：

硬件：PC 电脑一台，网络正常；

配置：windows 系统>=7，内存>=8G，硬盘 500G 及以上；

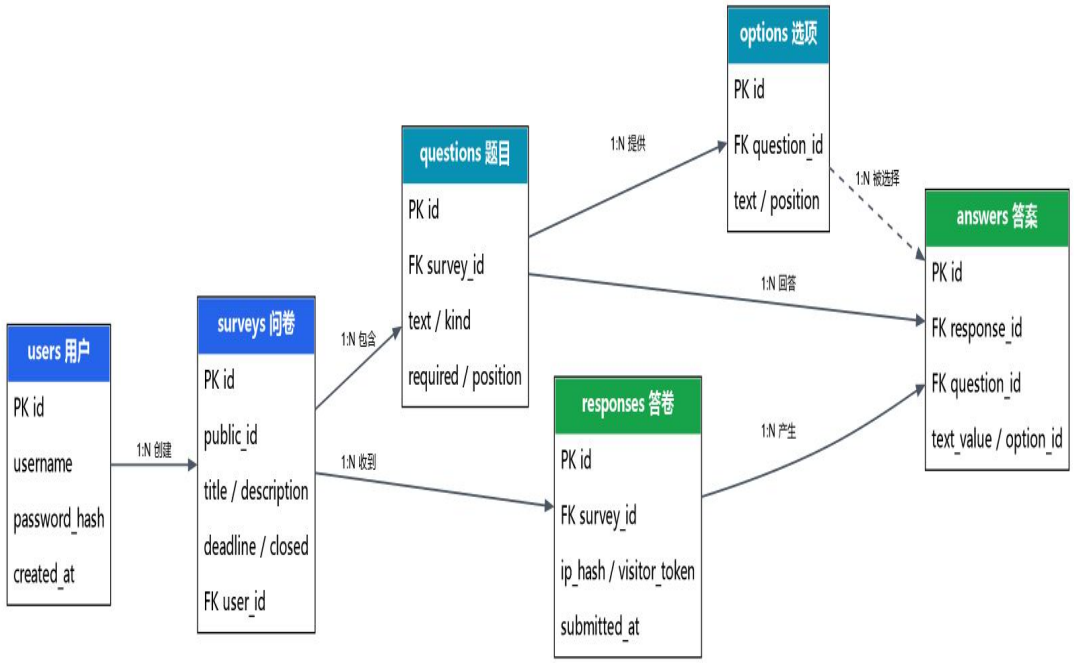
软件：Python+PyCharm+Flask+MySQL+Navicat。

四、实验内容与要求：

（请填写：① 选题路线：☐ 路线 A 课堂项目延伸——项目名称：_____；☑ 路线 B 题库选题——题目编号及名称：题目一：在线投票与问卷系统 。② 项目简介与功能清单：简述项目完成了哪些功能模块，课堂原有功能与自己新增/扩展的功能请分别列出。）
项目简介：本项目基于 Flask、MySQL、Jinja2 和 ECharts，实现问卷创建、匿名填写、结果统计和 CSV 导出。
功能模块：用户注册登录；问卷创建、编辑、关闭、删除；单选题、多选题和填空题；截止时间控制；填写链接生成；必填校验；重复提交限制；实时统计和 Excel 兼容 CSV 导出。
扩展功能：使用 IP 哈希与 Cookie/账号标识进行防重复；统计页同时提供单选题柱状图、饼图和多选题横向条形图。

一、报告正文（以下为评分项，请逐项填写）：

1、系统架构与数据库设计（ER 图、表结构说明）



系统采用 Flask + Jinja2 的前后端不分离架构。浏览器通过路由访问登录、问卷管理、填写和统计页面，Flask 负责请求处理、权限校验和模板渲染，SQLAlchemy 负责 MySQL 数据访问，ECharts 负责统计结果可视化。

ER 关系：User 与 Survey 为一对多；Survey 与 Question 为一对多；Question 与 Option 为一对多；Survey 与 Response 为一对多；Response 与 Answer 为一对多；Answer 可关联 Option，也可保存填空文本。

主要表结构：users(id, username, password_hash, created_at)；surveys(id, public_id, title, description, deadline, closed, user_id)；questions(id, survey_id, text, kind, required, position)；options(id, question_id, text, position)；responses(id, survey_id, ip_hash, visitor_token, submitted_at)；answers(id, response_id, question_id, text_value, option_id)。

核心数据流：创建者登录后创建问卷和题目，系统生成 public_id 填写链接；填写者提交答案后写入 responses 和 answers；统计端按 option_id 使用 GROUP BY 和 COUNT 实时聚合，最终由 ECharts 展示或导出 CSV。

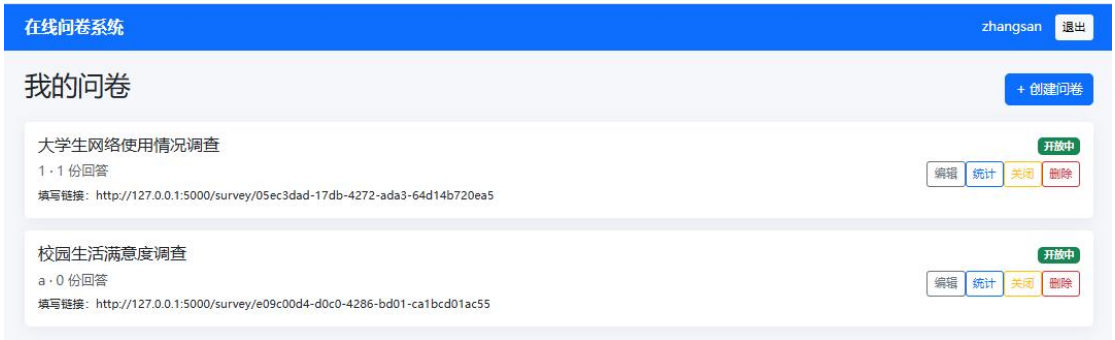
2、核心功能实现说明（附关键代码与运行截图）

系统使用 Flask Session 保存登录状态。用户注册时使用 Werkzeug 对密码进行哈希处理，登录时通过密码哈希校验身份。创建问卷、编辑问卷、关闭问卷、删除问卷和查看统计等管理功能均通过 login_required() 检查登录状态。

创建者登录后可以设置问卷标题、说明和截止时间，并动态添加题目。系统支持单选题、多选题和填空题，题目和选项分别保存到 questions 和 options 表。每份问卷使用唯一的 public_id 生成填写链接





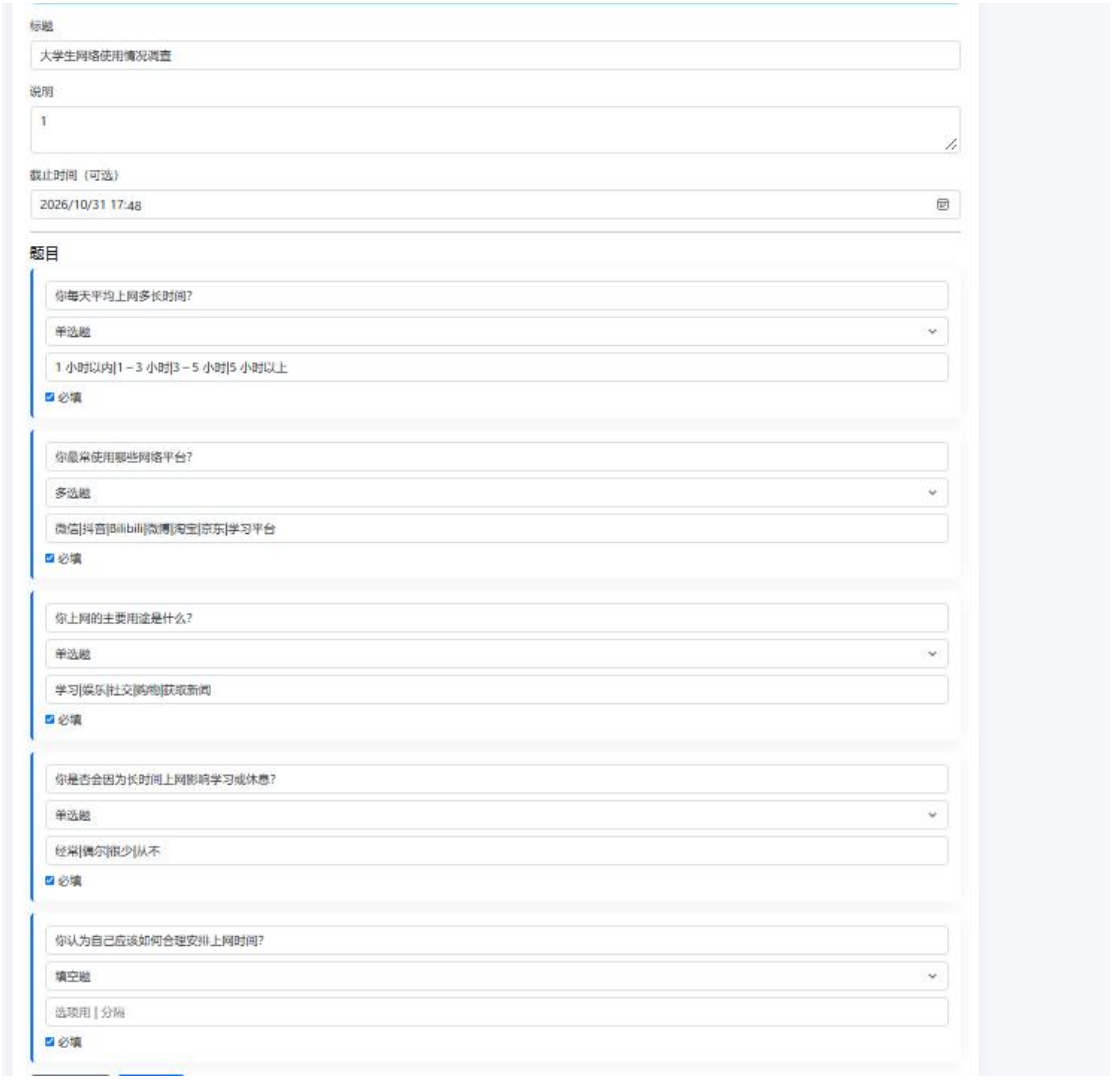



```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form.get('username', '').strip()
        password = request.form.get('password', '')
        if not username or len(password) < 6:
            flash('用户名不能为空，密码至少 6 位。', 'danger')
        elif User.query.filter_by(username=username).first():
            flash('用户名已存在。', 'danger')
        else:
            db.session.add(User(username=username,
password_hash=generate_password_hash(password)))
            db.session.commit()
            flash('注册成功，请登录。', 'success')
```

```
            return redirect(url_for('login'))
    return render_template('auth.html', mode='register')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = User.query.filter_by(username=request.form.get('username', '').strip()).first()
        if user and check_password_hash(user.password_hash, request.form.get('password', '')):
            session['user_id'] = user.id
            return redirect(request.args.get('next') or url_for('dashboard'))
        flash('用户名或密码错误。', 'danger')
    return render_template('auth.html', mode='login')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('index'))
```






```
class Survey(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    public_id = db.Column(db.String(36), unique=True, nullable=False, default=lambda:
str(uuid.uuid4()))
    title = db.Column(db.String(200), nullable=False)
    description = db.Column(db.Text, default='')
    deadline = db.Column(db.DateTime, nullable=True)
    closed = db.Column(db.Boolean, default=False, nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey('user.id'), nullable=False)
    questions = db.relationship('Question', backref='survey', cascade='all, delete-orphan',
order_by='Question.position')
    responses = db.relationship('Response', backref='survey', cascade='all, delete-orphan')
class Question(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    survey_id = db.Column(db.Integer, db.ForeignKey('survey.id'), nullable=False)
    text = db.Column(db.Text, nullable=False)
    kind = db.Column(db.String(20), nullable=False)
    required = db.Column(db.Boolean, default=True, nullable=False)
    position = db.Column(db.Integer, default=0, nullable=False)
    options = db.relationship('Option', backref='question', cascade='all, delete-orphan',
order_by='Option.position')


class Option(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    question_id = db.Column(db.Integer, db.ForeignKey('question.id'), nullable=False)
    text = db.Column(db.String(200), nullable=False)
    position = db.Column(db.Integer, default=0, nullable=False)
@app.route('/survey/new', methods=['GET', 'POST'])
def create_survey():
    login_redirect = login_required()
    if login_redirect: return login_redirect
    if request.method == 'POST':
        survey = Survey(title=request.form.get('title', '').strip(),
description=request.form.get('description', '').strip(), user_id=session['user_id'])
        deadline = request.form.get('deadline', '').strip()
        if deadline:
            survey.deadline = datetime.fromisoformat(deadline)
        if not survey.title:
```

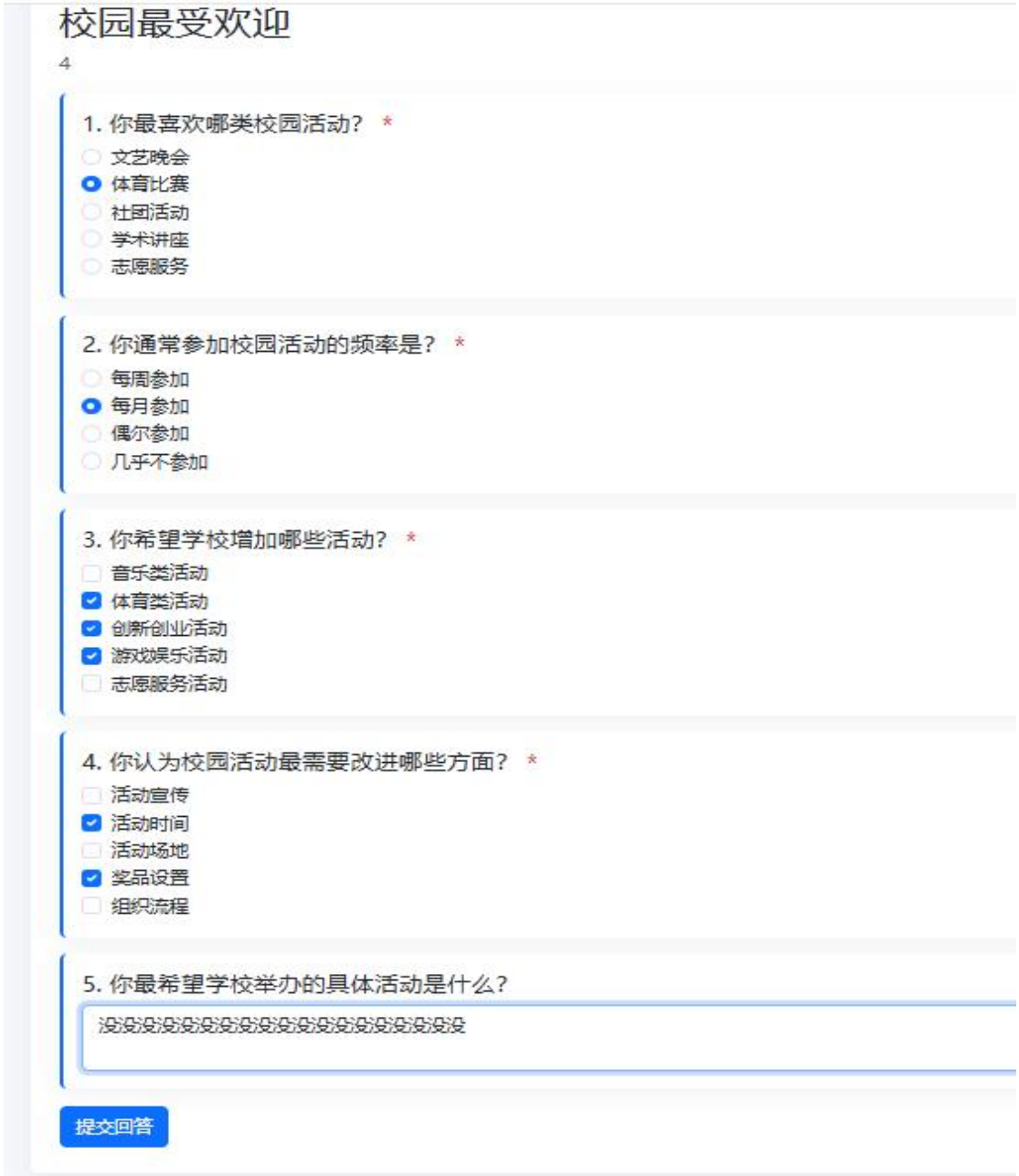
```
                flash('标题不能为空。', 'danger')
                return render_template('survey_form.html', survey=None)
            db.session.add(survey)
            db.session.flush()
            texts = request.form.getlist('question_text')
            kinds = request.form.getlist('question_kind')
            requireds = request.form.getlist('required_question')
            options = request.form.getlist('question_options')
            for i, text in enumerate(texts):
                if not text.strip(): continue
                q = Question(survey_id=survey.id, text=text.strip(), kind=kinds[i], required=str(i) in
requireds, position=i)
                db.session.add(q)
                db.session.flush()
                for j, option_text in enumerate(options[i].split('|') if i < len(options) else []):
                    if option_text.strip(): db.session.add(Option(question_id=q.id,
text=option_text.strip(), position=j))
            db.session.commit()
            flash('问卷创建成功。', 'success')


{% extends 'base.html' %}{% block content %}
<div class="card p-4"><h2>{{ '编辑问卷' if editing else '创建问卷' }}</h2>{% if editing and
survey.responses %}<div class="alert alert-info">该问卷已有答卷，保存时只更新标题、说明和截止
时间，题目与选项不会改变。</div>{% endif %}
<form method="post"><label class="form-label">标题</label><input class="form-control"
name="title" value="{{ survey.title if survey else '' }}" required>
<label class="form-label mt-3">说明</label><textarea class="form-control"
name="description">{{ survey.description if survey else '' }}</textarea>
<label class="form-label mt-3">截止时间（可选）</label><input class="form-control"
type="datetime-local" name="deadline" value="{{ survey.deadline.strftime('%Y-%m-%dT%H:%M') if
survey and survey.deadline else '' }}">
<hr><h5>题目</h5><div id="questions">{% if survey %}{% for q in survey.questions %}<div
class="question-card card p-3 mb-3"><input class="form-control mb-2" name="question_text"
value="{{q.text}}" required><select class="form-select mb-2" name="question_kind"><option
value="single" {% if q.kind=='single' %}selected{% endif %}>单选题</option><option
value="multiple" {% if q.kind=='multiple' %}selected{% endif %}>多选题</option><option
value="text" {% if q.kind=='text' %}selected{% endif %}>填空题</option></select><input
class="form-control mb-2" name="question_options"
value="{{q.options|map(attribute='text')|join('|')}}" placeholder="选项用 | 分隔"><label><input
type="checkbox" name="required_question" value="{{loop.index0}}" {% if q.required %}checked{%
endif %}> 必填</label></div>{% endfor %}{% endif %}</div>
<button type="button" class="btn btn-outline-secondary" onclick="addQuestion()">+ 添加题目
</button><button class="btn btn-primary ms-2">保存问卷</button></form></div>
```

```
<script>let n={{survey.questions|length if survey else 0}};function addQuestion(){let i=n++;document.querySelector('#questions').insertAdjacentHTML('beforeend',`<div class="question-card card p-3 mb-3"><input class="form-control mb-2" name="question_text" placeholder="题目内容" required><select class="form-select mb-2" name="question_kind"><option value="single">单选题</option><option value="multiple">多选题</option><option value="text">填空题</option></select><input class="form-control mb-2" name="question_options" placeholder="选项用 | 分隔，如：非常满意|满意|一般"><label><input type="checkbox" checked name="required_question" value="${i}"> 必填</label></div>`)}{% if not survey %}addQuestion(){% endif %}</script>
{% endblock %}


{% extends 'base.html' %}{% block content %}
<div class="d-flex justify-content-between align-items-center mb-3"><h2>我的问卷</h2><a class="btn btn-primary" href="{{ url_for('create_survey') }}">+ 创建问卷</a></div>
{% for survey in surveys %}<div class="card p-3 mb-3"><div class="d-flex justify-content-between"><div><h5>{{survey.title}}</h5><p class="text-muted mb-1">{{survey.description or '无说明'}} · {{survey.responses|length}} 份回答</p><small>填写链接：{{url_for('fill_survey',public_id=survey.public_id,_external=True)}}</small></div><div class="text-end"><span class="badge bg-{{'success' if survey.available else 'secondary'}}">{{'开放中' if survey.available else '已结束'}}</span><div class="mt-2">{% if survey.available %}<a class="btn btn-sm btn-outline-secondary" href="{{url_for('edit_survey',public_id=survey.public_id)}}">编辑</a>{% endif %}<a class="btn btn-sm btn-outline-primary" href="{{url_for('results',public_id=survey.public_id)}}">统计</a>{% if survey.available %}<form class="d-inline" method="post" action="{{url_for('close_survey',public_id=survey.public_id)}}"><button class="btn btn-sm btn-outline-warning">关闭</button></form>{% endif %}<form class="d-inline" method="post" action="{{url_for('delete_survey',public_id=survey.public_id)}}"><button class="btn btn-sm btn-outline-danger">删除</button></form></div></div></div></div>{% else %}<div class="alert alert-info">还没有自己创建的问卷。</div>{% endfor %}
<h2 class="mt-5">可填写问卷</h2>
{% for survey in public_surveys %}<div class="card p-3 mb-3"><div class="d-flex justify-content-between"><div><h5>{{survey.title}}</h5><p class="text-muted mb-1">{{survey.description or '无说明'}} · {{survey.responses|length}} 份回答</p></div><a class="btn btn-primary align-self-center" href="{{url_for('fill_survey',public_id=survey.public_id)}}">开始填写</a></div></div>{% else %}<div class="alert alert-info">暂无开放中的问卷。</div>{% endfor %}
{% endblock %}
```

填写者无需登录即可访问问卷链接。服务端会检查问卷是否关闭或超过截止时间，并验证必填题和选项合法性。登录填写者使用账号标识，匿名填写者使用 Cookie 标识，避免同一填写者重复提交。提交成功后系统显示“感谢参与”。



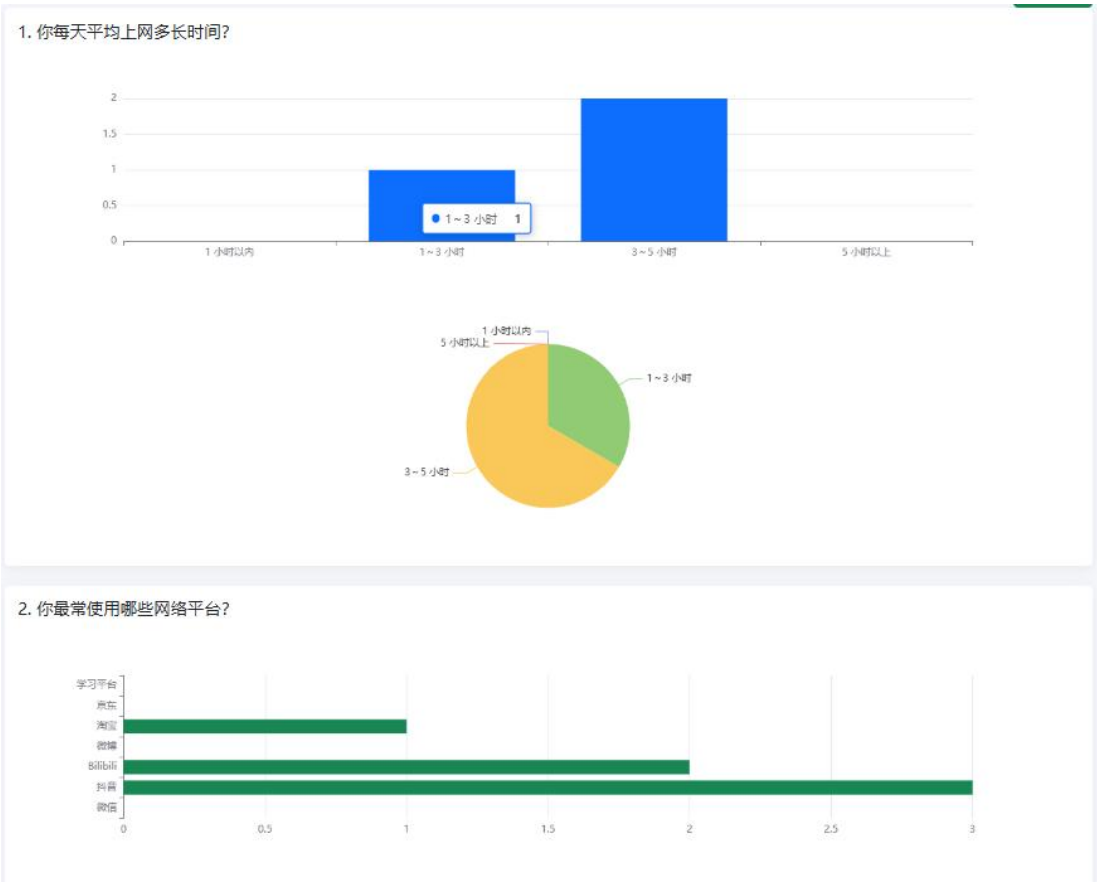




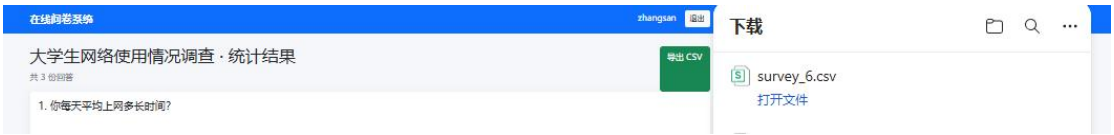

```
@app.route('/survey/<public_id>', methods=['GET', 'POST'])
def fill_survey(public_id):
    survey = Survey.query.filter_by(public_id=public_id).first_or_404()
    cookie_token = request.cookies.get('visitor_token') or uuid.uuid4().hex
    token = f'user:{session["user_id"]}' if session.get('user_id') else f'guest:{cookie_token}'
    ip_hash = hashlib.sha256(request.remote_addr.encode()).hexdigest()
    if request.method == 'POST':
        if not survey.available: return render_template('message.html', title='问卷已结束',
message='该问卷已关闭或超过截止时间。'), 410
        if Response.query.filter_by(survey_id=survey.id, visitor_token=token).first():
            return render_template('message.html', title='请勿重复提交', message='同一填写者只
能提交一次。'), 409
        response = Response(survey_id=survey.id, ip_hash=ip_hash, visitor_token=token)
        db.session.add(response)
        db.session.flush()
        for question in survey.questions:
            values = request.form.getlist(f'question_{question.id}')
            if question.required and not any(v.strip() for v in values):
```

```
                    db.session.rollback()
                    flash(f'请填写：{question.text}', 'danger')
                    return render_template('fill.html', survey=survey)
                for value in values:
                    if question.kind == 'text': db.session.add(Answer(response_id=response.id,
question_id=question.id, text_value=value.strip()))
                    else:
                        option = Option.query.filter_by(id=value, question_id=question.id).first()
                        if option: db.session.add(Answer(response_id=response.id,
question_id=question.id, option_id=option.id))
        db.session.commit()
        result = render_template('message.html', title='感谢参与', message='你的回答已成功提交。
')
        response = Response.query.get(response.id)
        resp = FlaskResponse(result)
        resp.set_cookie('visitor_token', cookie_token, max_age=31536000, httponly=True,
samesite='Lax')
        return resp
    return render_template('fill.html', survey=survey)
. . . . . .
{% extends 'base.html' %}{% block content %}<div class="card p-4"><h2>{{survey.title}}</h2><p
class="text-muted">{{survey.description}}</p><form method="post" onsubmit="return
validateSurvey()">{% for q in survey.questions %}<div class="question-card card p-3
mb-3"><h5>{{loop.index}}. {{q.text}} {% if q.required %}<span class="text-danger">*</span>{%
endif %}</h5>{% if q.kind=='text' %}<textarea class="form-control" name="question_{{q.id}}" {% if
q.required %}required{% endif %}></textarea>{% else %}{% for o in q.options %}<div
class="form-check"><input class="form-check-input" type="{{'radio' if q.kind=='single' else
'checkbox'}}" name="question_{{q.id}}" value="{{o.id}}" {% if q.required and
q.kind=='single' %}required{% endif %}><label class="form-check-label">{{o.text}}</label></div>{%
endfor %}{% endif %}</div>{% endfor %}<button class="btn btn-primary">提交回答
</button></form></div><script>function validateSurvey(){let
valid=true;document.querySelectorAll('input[type=checkbox][data-required]').forEach(()=>{});{% for
q in survey.questions if q.kind=='multiple' and
q.required %}if(!document.querySelectorAll('input[name="question_{{q.id}}"]:checked').length){alert
('请至少选择一项：{{q.text|e}}');valid=false;}{% endfor %}return valid;}</script>{% endblock %}
```

统计数据直接从数据库实时聚合。系统使用 SQLAlchemy 构造 GROUP BY + COUNT 查询，按选项统计被选择次数。单选题使用柱状图和饼图展示，多选题使用横向条形图展示，填空题以文本列表显示。








```
@app.route('/survey/<public_id>/results')
def results(public_id):
    login_redirect = login_required()
    if login_redirect: return login_redirect
    survey = owner_survey(public_id)
    result_data = []
    for q in survey.questions:
        rows = []
        if q.kind != 'text':
            counts = dict(
```

```
                    db.session.query(Answer.option_id, db.func.count(Answer.id))
                    .filter(Answer.question_id == q.id)
                    .group_by(Answer.option_id)
                    .all()
                )
                rows = [{'name': o.text, 'value': counts.get(o.id, 0)} for o in q.options]
            else:
                rows = [{'text': a.text_value} for a in Answer.query.filter_by(question_id=q.id).all() if
a.text_value]
            result_data.append({'id': q.id, 'text': q.text, 'kind': q.kind, 'rows': rows})
        return render_template('results.html', survey=survey, result_data=result_data,
response_count=len(survey.responses))


{% extends 'base.html' %}{% block content %}<div class="d-flex
justify-content-between"><div><h2>{{survey.title}} • 统计结果</h2><p class="text-muted">共
{{response_count}} 份回答</p></div><a class="btn btn-success"
href="{{url_for('export_csv',public_id=survey.public_id)}}">导出 CSV</a></div>{% for item in
result_data %}<div class="card p-3 mb-4"><h5>{{loop.index}}. {{item.text}}</h5>{% if
item.kind!='text' %}<div id="bar-{{item.id}}" style="height:300px"></div>{% if
item.kind=='single' %}<div id="pie-{{item.id}}" style="height:300px"></div>{%
endif %}<script>document.addEventListener('DOMContentLoaded',()=>{let
names={{item.rows|map(attribute='name')|list|tojson}},values={{item.rows|map(attribute='value')|list
|tojson}},bar=echarts.init(document.getElementById('bar-{{item.id}}'));{% if
item.kind=='multiple' %}bar.setOption({tooltip:{},yAxis:{type:'category',data:names},xAxis:{type:'val
ue'},series:[{type:'bar',data:values,itemStyle:{color:'#198754'}}]});{%
else %}bar.setOption({tooltip:{},xAxis:{type:'category',data:names},yAxis:{type:'value'},series:[{type:
'bar',data:values,itemStyle:{color:'#0d6efd'}}]});let
pie=echarts.init(document.getElementById('pie-{{item.id}}'));pie.setOption({tooltip:{},series:[{type:'pi
e',radius:'65%',data:names.map((name,i)=>({name:name,value:values[i]}))}]});{%
endif %}});</script>{% else %}<ul>{% for row in item.rows %}<li>{{row.text}}</li>{% else %}<li
class="text-muted">暂无填空答案</li>{% endfor %}</ul>{% endif %}</div>{% endfor %}<script
src="https://cdn.jsdelivr.net/npm/echarts@5/dist/echarts.min.js"></script>{% endblock %}
```

系统支持将问卷统计结果导出为 CSV 文件。导出内容包含题目、选项或答案以及对应数量。文件开头加入 UTF-8 BOM，使用 Excel 打开时可以正常显示中文。



	A	B	C	D	E
1	题目	选项/答案	数量		
2	你每天平均上网多长时间？	1 小时以内	0		
3	你每天平均上网多长时间？	1~3 小时	1		
4	你每天平均上网多长时间？	3~5 小时	2		
5	你每天平均上网多长时间？	5 小时以上	0		
6	你最常使用哪些网络平台？	微信	0		
7	你最常使用哪些网络平台？	抖音	3		
8	你最常使用哪些网络平台？	Bilibili	2		
9	你最常使用哪些网络平台？	微博	0		
10	你最常使用哪些网络平台？	淘宝	1		
11	你最常使用哪些网络平台？	京东	0		
12	你最常使用哪些网络平台？	学习平台	0		
13	你上网的主要用途是什么？	学习	1		
14	你上网的主要用途是什么？	娱乐	0		
15	你上网的主要用途是什么？	社交	2		
16	你上网的主要用途是什么？	购物	0		
17	你上网的主要用途是什么？	获取新闻	0		
18	你是否会因为长时间上网影响学习或休息？	经常	0		
19	你是否会因为长时间上网影响学习或休息？	偶尔	1		
20	你是否会因为长时间上网影响学习或休息？	很少	1		
21	你是否会因为长时间上网影响学习或休息？	从不	1		
22	你认为自己应该如何合理安排上网时间？	1	1		
23	你认为自己应该如何合理安排上网时间？	1	1		
24	你认为自己应该如何合理安排上网时间？	2	1		
25					
26					

```
@app.route('/survey/<public_id>/export.csv')
def export_csv(public_id):
    login_redirect = login_required()
    if login_redirect: return login_redirect
    survey = owner_survey(public_id)
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(['题目', '选项/答案', '数量'])
    for q in survey.questions:
        if q.kind == 'text':
            for answer in Answer.query.filter_by(question_id=q.id).all(): writer.writerow([q.text,
answer.text_value or '', 1])
        else:
            for option in q.options: writer.writerow([q.text, option.text,
Answer.query.filter_by(question_id=q.id, option_id=option.id).count()])
    return FlaskResponse('\ufeff' + output.getvalue(), mimetype='text/csv; charset=utf-8',
headers={'Content-Disposition': f'attachment; filename=survey_{survey.id}.csv'})
```

```
with app.app_context():
    db.create_all()
return app
```

3、结果分析和总结（如课设达到的预期效果，是否按时完成，遇到哪些问题？如何解决的？还需要学习和提高的地方？下一步的计划和打算？）

本项目已完成预期的在线投票与问卷功能，能够支持问卷创建、匿名填写、必填校验、截止时间控制、重复提交限制、统计图表和 CSV 导出。系统已在本地 Flask + MySQL 环境中完成主要流程验证，并准备了 demo 测试账号和 12 条测试答卷。

开发过程中遇到的主要问题包括多选题 required 属性导致必须全选、同一 IP 下不同用户被误判重复、CSV 响应类命名冲突、已有答卷时编辑题目触发外键约束等。通过调整前端校验逻辑、改用账号或 Cookie 标识、区分 FlaskResponse 与数据库 Response 模型，以及保留已有答卷的题目结构，分别解决了这些问题。

仍需提高的方面包括完善权限角色、增加问卷版本管理、补充自动化测试和优化前端交互。下一步计划是补充运行截图、完善设计文档中的图示，并继续进行多人并发和异常场景测试。

二、代码仓库链接与 AI 协作说明（必填）：

代码仓库链接（GitHub）：https://github.com/L-YH2024/py

AI 工具使用情况说明：使用 Codex 进行需求分析、Flask 项目搭建、数据库模型设计、路由与模板编写、Bug 定位修复、统计 SQL 优化、README 和课程设计文档整理。AI 协作记录已整理在项目中的 AI_COLLABORATION_LOG.md 文件中。